

15th Progress Report

W. Chan

Date:	05/06/2024
Subject:	RL for Flying-V
Supervisors:	Dr. E.J. van Kampen
Project term:	01/2024 - 08/2024
E-mail:	w.y.chan@student.tudelft.nl

Outcome of the previous meeting

- Looked over changes made to thesis document (wording changes, addition of statistical testing, graph presentation changes)
- Discussed Simulink to Python pipeline, decided to recompile Simulink with the table3.c file.
- Reviewed test flight manoeuvre and the stages of test flight to use:
 1. Try decoupled pitching and rolling rate control
 2. If 1. successful, try more coupled pitching and rolling rate control
 3. If 2. successful, try pitching and rolling attitude control.
- Had a couple of miscellaneous questions left unanswered due to time.

Description of the results since last meeting

- **Recompiled CitAST** with the correct MATLAB directories, now setup.py is able to find the table3.c file to use for the S function.
- **Redone verification** of python executable. Exported .csv file of python executable response to trim input and found that there was zero (**zero**) difference between the Simulink and python CitAST executables! As shown in Figure 1. Despite the suspiciously flat and perfect-looking plots in Figure 1, double checking shows that the model response between the .pyd executable and Simulink are, in fact, identical.
- **Wrestled with expanding IDHP implementation**

Struggled greatly with trying to generalize the pre-existing IDHP implementation to allow a choice between longitudinal/lateral controller split, and having only one controller, but realised that this made the algorithm very ugly and illegible. Thus decided to simply create a new class that explicitly splits the algorithm into a longitudinal and a lateral half.
- **Decided to implement network Jacobian updates later, but I'd like to discuss future steps.**

Realised it was wiser to implement a minimum-working-order (MWO) of IDHP before deriving and implementing network Jacobians for network updates. Therefore, decided to start implementing IDHP using TensorFlow's auto-diff functions and network SGD optimizers. One idea here now is to get this MWO to:

1. Demonstrate satisfactory **longitudinal** rate tracking performance.
2. Given success in 1., to make it work for **lateral** rate tracking until satisfactory.
3. Given success in 2., to use more complicated/longer flight manoeuvres (overlapping rolling and pitching control, extended/larger magnitude rolling and pitching...)
4. Given success in 3., to try out attitude tracking.

As of writing this progress note, IDHP shows *almost* satisfactory performance is achieved in step 1. (longitudinal rate tracking), see Figure 2. Further tuning seems to indicate that the policy is very prone to becoming unstable, this observation is shown in Figure 3. Due diligence is done here, the IDHP controller is replaced by a static Proportional controller and a similar result is observed when the proportional gain is set too high, this is shown in Figure 4, this indicates that the oscillatory/unstable controller result is not a fault of the IDHP algorithm, but rather a fault of the controller system design.

Additionally, with the total sum of IDHP/RL coding experience I've gained so far, managed to get Stefan Heyer's IDHP implementation completely working, and found that with his MDP state space, which encompasses several longitudinal states, e.g. for the longitudinal controller $s = [q \quad \alpha \quad \theta \quad q - q_{ref}]$, one can indeed get almost perfect tracking performance.

Therefore, with these results and observations, and comparing them to previous students (Stefan and Jun) results, another alternative route forward is also posed. This alternative route entails only one change to the afore-listed steps, that is to use a different set of MDP states. Whereas right now, the MDP state is only tracking error, e.g. longitudinal controller's MDP state is $q - q_{ref}$, I could simply adopt Stefan's MDP state space choice.

This is essentially repeating an experiment I have done in research phase one, where I tested using 4 possible MDP state combinations. The difference now would be that I will spend more time doing hyperparameter tuning for the alternative MDP states.

This does, however, bring added time pressure. Currently, there is at most 2 more full working week's worth of time I could dedicate to getting this MWO IDH to work, and implementing the Jacobian updates. If I were to do all four steps for the MWO, then I am not very certain I can implement the Jacobian/eligibility trace updates and tuning hyperparameters thereafter. This is before considering the time needed to implement and tune for multistep updates.

Thus, I would like to discuss what would be a wise way to proceed (Limit scope to rate tracking? Limit scope to only longitudinal tracking? Limit scope to only attitude tracking? Don't experiment with this other MDP state space?)

- **Running current IDHP on the previously designed flight manoeuvre (only longitudinal)**

Experimented with trying out the previously proposed flight manoeuvres on the current implementation of IDHP, but only the pitching task. As a reminder, the selected task was to first pitch up for a couple of seconds, then return the pitch to zero, then pitch down for a couple of seconds, and finally return the pitch to zero. But looking at the results, it seems wise to add in a warmup manoeuvre at the beginning to get better tracking performance. See Figure 5.

- **Did some thinking about the state space which the RLS estimated model needs to span.**

Despite some previous students using subsets of the CitAST state space as the state space for the RLS estimated model, I hypothesize that it is also just as valid to incorporate tracking error as part of this state space and allow RLS to estimate a model spanning this additional dimension. Bringing this hypothesis further, with the current MDP state space only spanning the tracking error, it would be interesting to see how the algorithm behaves when the RLS model state space **only** spans tracking error, instead of spanning say α or q .

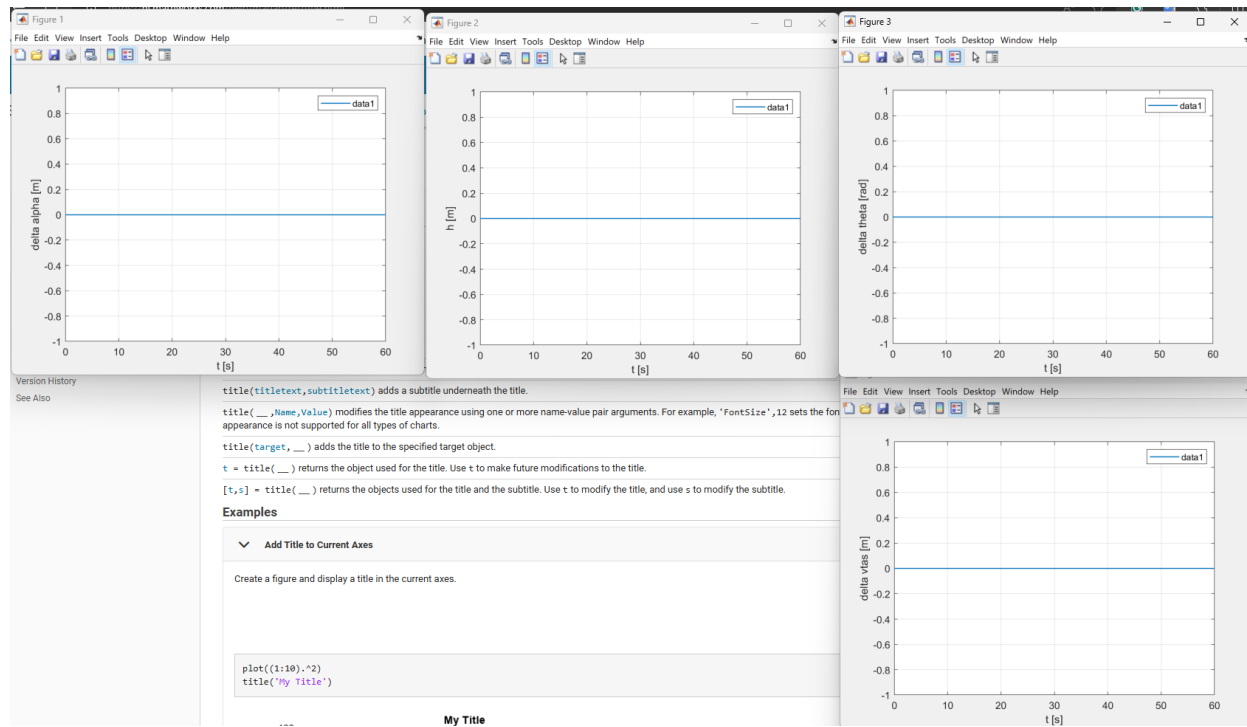
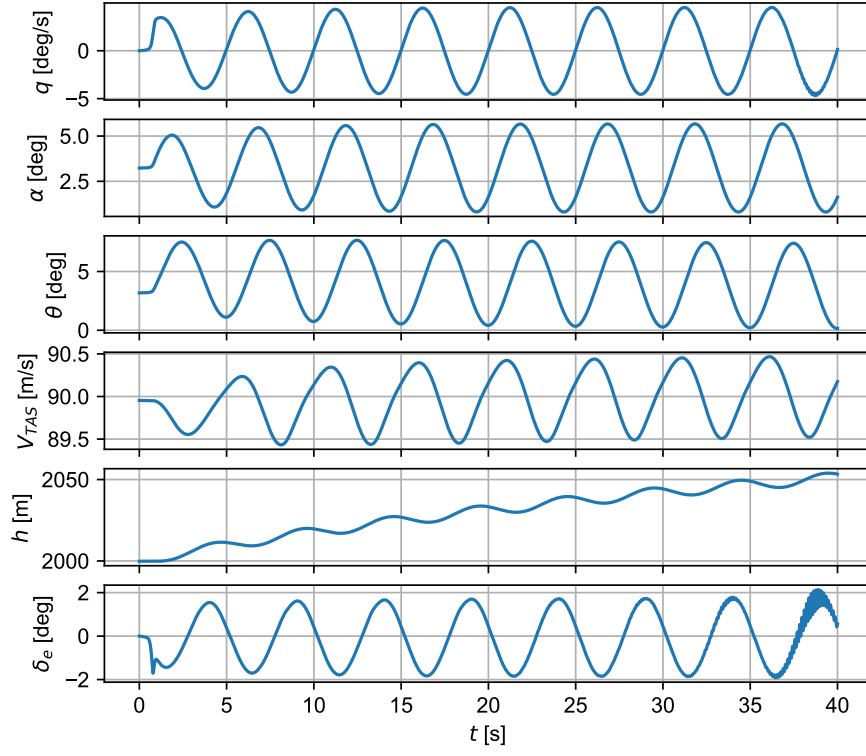


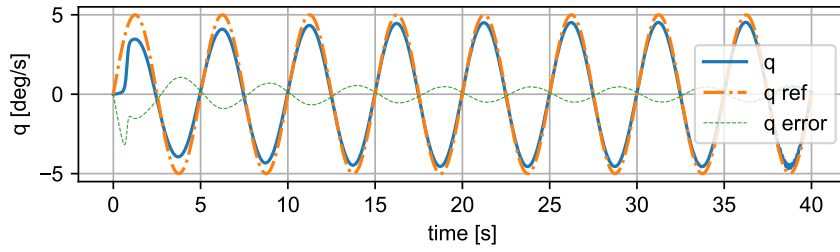
Figure 1: Plot of difference between state responses of .pyd and Simulink CitAST models for 4 longitudinal states.

Plans for next 2 weeks

1. Experiment with the different MDP state space choices, and perform dedicated hyperparameter tuning to see if I can replicate Stefan's results.
2. Proceed with making MWO IDHP work, and then implement Jacobian updates to enable using eligibility traces, then finally implement multi-step updates.

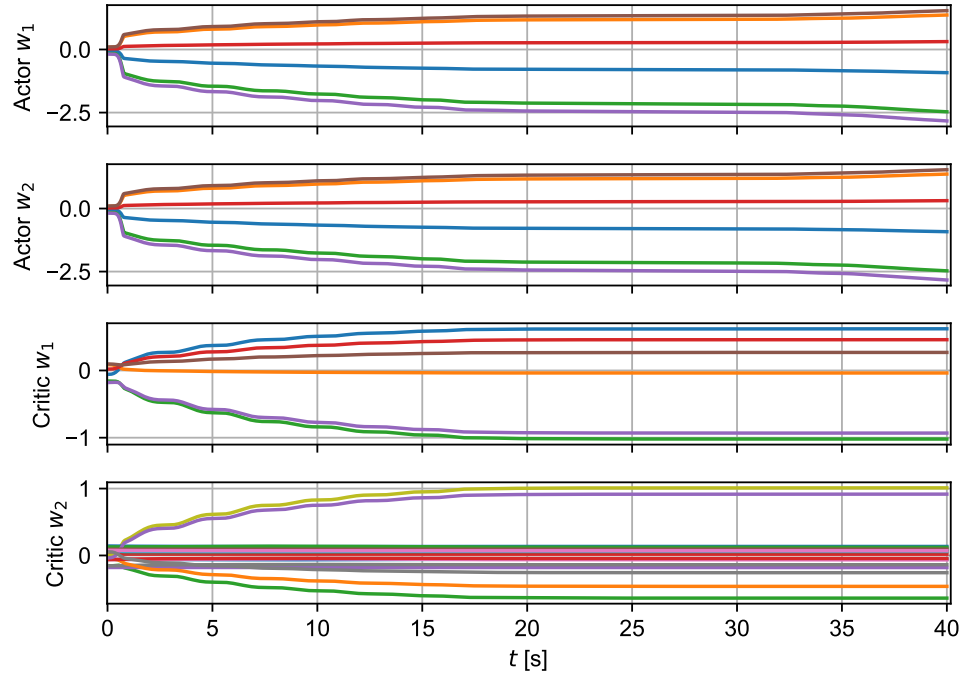


(a)

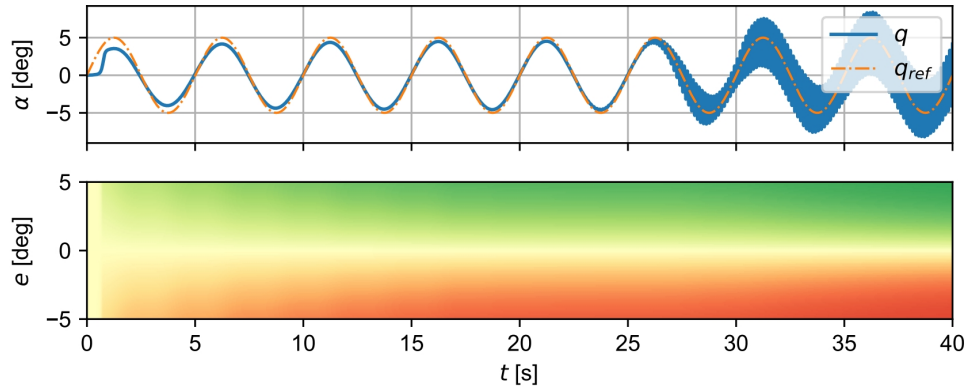


(b)

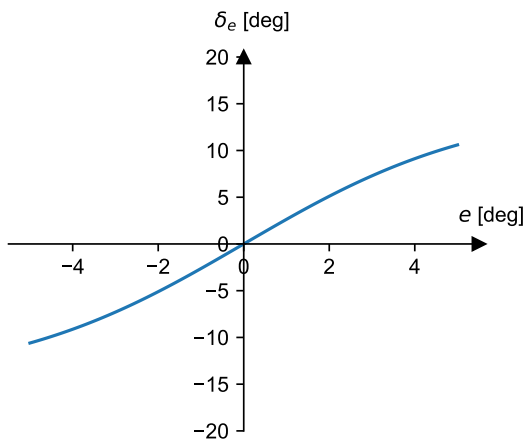
Figure 2: Current best q tracking performance achieved.



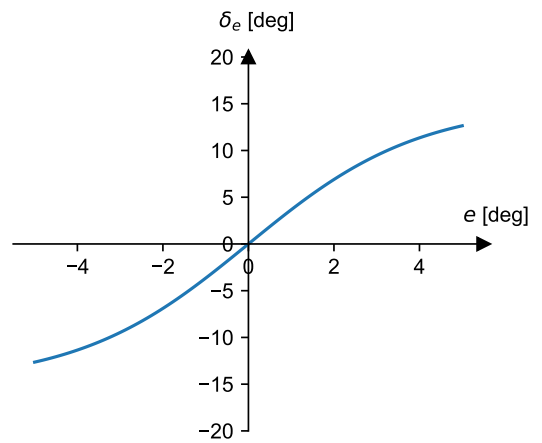
(a)



(b)



(c) Policy at 19.5 s.



(d) Policy at 35 s.

Figure 3: Response of a noisy/unstable IDHP controller, note the actor weights do not dramatically change when the controller exhibits unstable action.

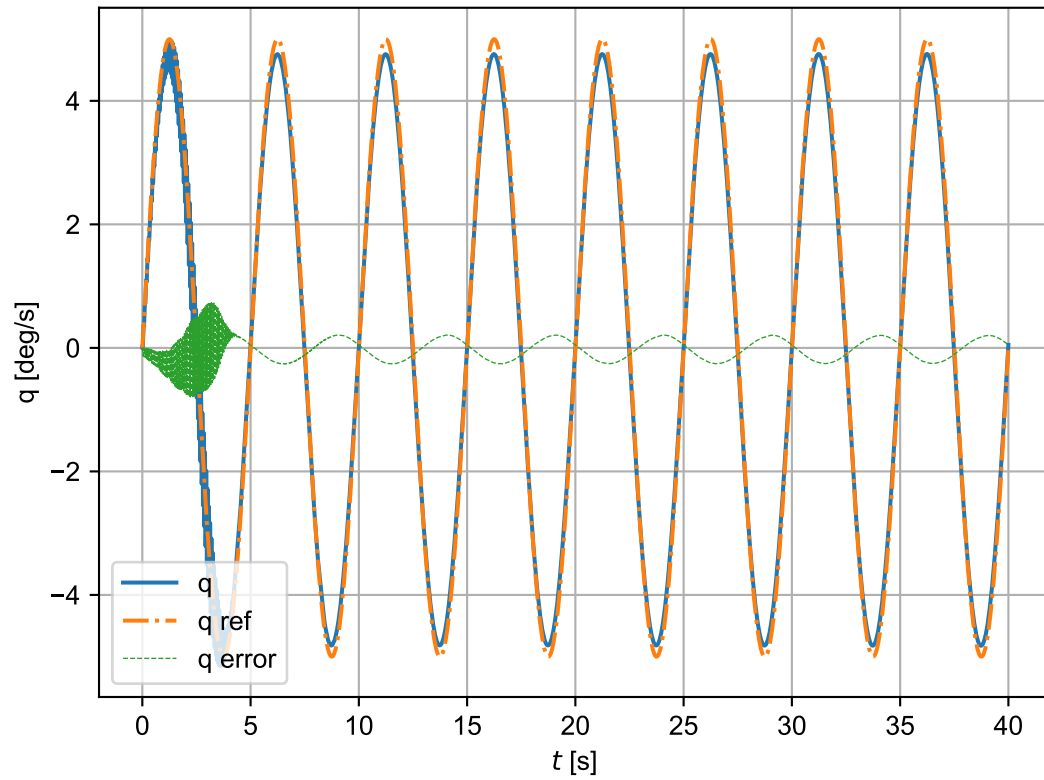


Figure 4: Static P gain controller shows unstable control action, P gain = 31.

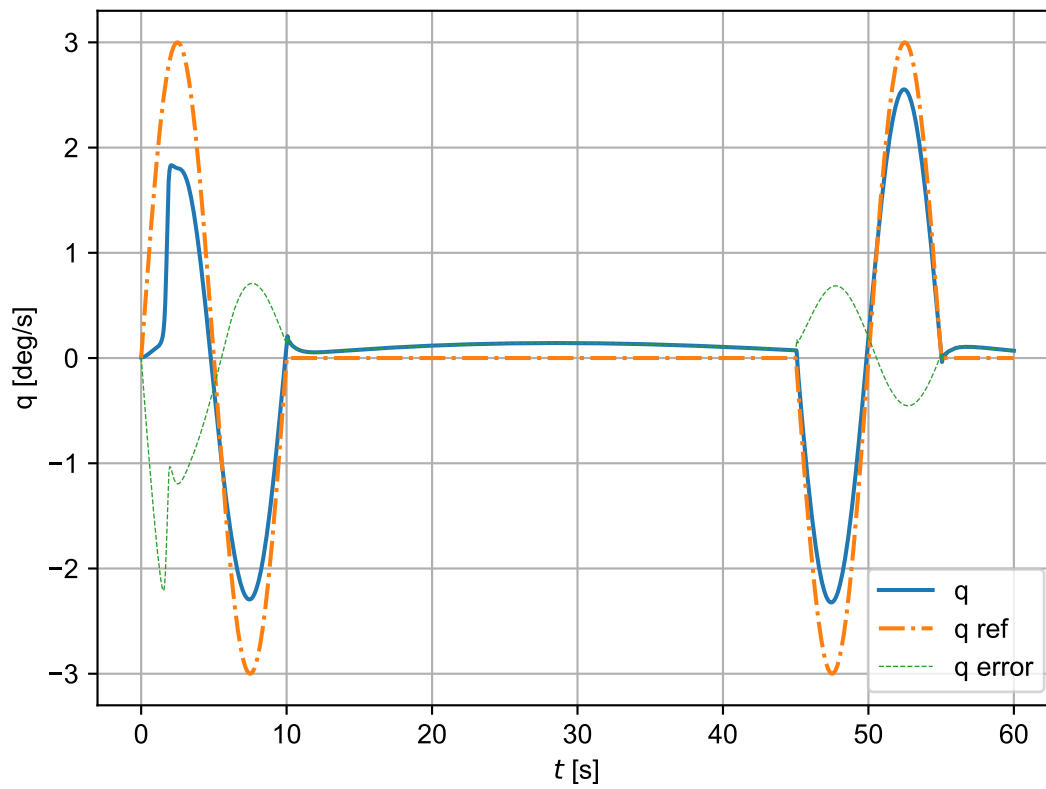


Figure 5: Testing on the previously proposed flight manoeuvre.